

Documento de Decisiones de Diseño y Modelado de Datos

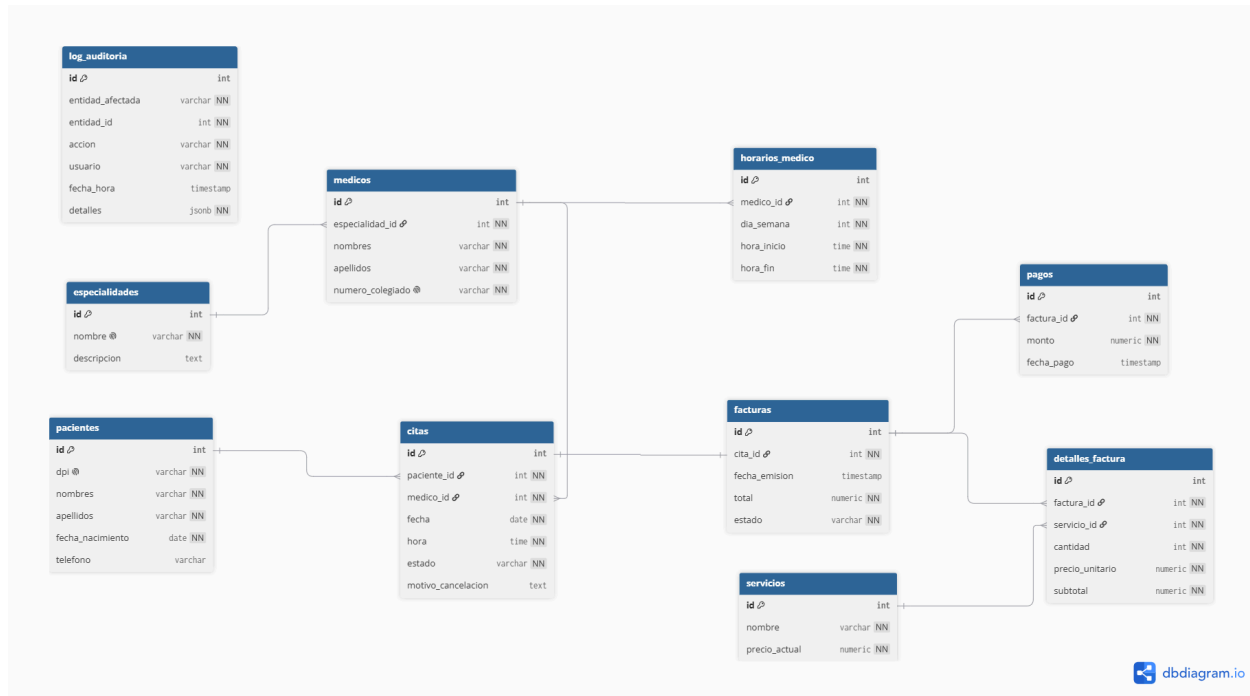
Proyecto Final - Opción A: Clínica Médica Privada

1. Arquitectura de Persistencia Políglota

El diseño de la base de datos para la clínica médica emplea una arquitectura de persistencia políglota, delegando responsabilidades específicas a dos motores distintos según la naturaleza de los datos.

- PostgreSQL (Datos Transaccionales): Se seleccionó para gestionar las entidades financieras, operativas y de agendamiento (Citas, Facturación, Pagos, Médicos y Pacientes). Estas operaciones requieren garantías ACID estrictas para evitar anomalías críticas, como la doble asignación de una cita o inconsistencias en los saldos de los pacientes.
- MongoDB (Datos Flexibles): Se designó exclusivamente para el almacenamiento de los `historiales_clinicos`. La justificación técnica radica en el Requerimiento Funcional 07 (RF-07), el cual especifica que la estructura de la información médica varía drásticamente según la especialidad. Utilizar un motor relacional para esto obligaría a emplear el antipatrón EAV (Entity-Attribute-Value) o a mantener tablas con exceso de valores nulos. MongoDB permite un diseño *schemaless* controlado que se adapta a cada consulta.

2. Modelado Relacional y Restricciones a Nivel de Esquema (PostgreSQL)



El esquema relacional fue normalizado hasta la Tercera Forma Normal (3FN) para evitar redundancias. Las entidades principales se dividieron en dos dominios lógicos: Flujo de Citas/Agenda y Flujo de Facturación/Pagos.

Para garantizar la consistencia de los datos directamente en el motor, se implementaron las siguientes restricciones (*Constraints*) basadas en las Reglas de Negocio (RN) y Requerimientos Funcionales (RF):

- Control de Solapamiento (RN-01): Se aplicó una restricción UNIQUE (*medico_id*, *fecha*, *hora*) en la tabla *citas* para prevenir que el motor inserte dos citas concurrentes para un mismo médico.
- Validación de Disponibilidad y Bloques de Tiempo: Para el cálculo de los horarios libres (RC-05) y la validación de solapamiento, se estableció una decisión de diseño a nivel de base de datos asumiendo una duración fija de 30 minutos por cita. Esta duración estará definida en la lógica de la función *get_horarios_libres*, permitiendo mapear la disponibilidad sin necesidad de agregar una columna extra de duración en la tabla transaccional.
- Validación de Horarios (RN-02): En la tabla *horarios_medico*, se implementó un CHECK (*hora_inicio* < *hora_fin*) para asegurar la coherencia lógica temporal en la disponibilidad de los médicos.

- Integridad Financiera (RN-04): En las tablas servicios y pagos, se establecieron restricciones CHECK (precio_actual > 0) y CHECK (monto > 0) respectivamente, impidiendo el registro de valores negativos que corrompan el cálculo de saldos.
- Log de Auditoría Desacoplado (RF-12): La tabla log_auditoria se diseñó intencionalmente sin llaves foráneas (FOREIGN KEY) hacia las tablas operativas. Se empleó una relación polimórfica (entidad_afectada, entidad_id) y una columna JSONB para almacenar el estado del registro modificado. Esto garantiza la inmutabilidad del historial ante la eliminación en cascada de registros operativos y mejora el rendimiento de inserción (*append-only*).

3. Estructura de Documentos y Decisiones en MongoDB

Script de MongoDB:

```
{
  "_id": "ObjectId('...')",
  "cita_id": 1542, // Referencia fuerte a PostgreSQL (Índice Único)

  // -- CAMPOS DESNORMALIZADOS --
  // Almacenados aquí para evitar $lookups imposibles hacia PostgreSQL
  "paciente_id": 89,
  "medico_id": 12,
  "especialidad": "Cardiología", // String explícito para permitir $group en RC-07 y RC-08
  "fecha_consulta": "2026-06-15T10:30:00Z",

  // -- DATOS CLÍNICOS ESTRUCTURADOS --
  "datos_base": {
    "motivo_consulta": "Dolor en el pecho al hacer esfuerzo",
    "signos_vitales": {
      "presion_arterial_sistolica": 140,
      "presion_arterial_diastolica": 90,
      "frecuencia_cardiaca": 88,
      "temperatura": 37.1,
      "peso_kg": 85
    },
    "notas_adicionales": "Paciente refiere que el dolor se irradia al brazo izquierdo."
  },

  "diagnosticos": [
    { "codigo_cie10": "I20.9", "descripcion": "Angina de pecho, no especificada", "tipo":
"Presuntivo" }
  ],
}
```

```
"medicamentos_recetados": [  
  { "nombre": "Aspirina", "dosis": "100mg", "frecuencia": "Cada 24 horas", "duracion_dias": 30 }  
],  
  
"exámenes_solicitados": [  
  "Electrocardiograma de esfuerzo",  
  "Perfil lipídico completo"  
],  
  
// -- MAGIA SCHEMALESS (RF-07) --  
// Este nodo cambia totalmente de estructura según la 'especialidad'  
"datos_especificos_especialidad": {  
  "antecedentes_cardiovasculares": true,  
  "dolor_escal_1_a_10": 7,  
  "ritmo_cardiaco_regular": false  
}  
}
```

La colección `historiales_clinicos` utiliza un patrón de diseño híbrido (Atributos Fijos + Esquema Flexible).

- Estrategia de Referencias: Se optó por vincular el historial clínico a PostgreSQL mediante una referencia fuerte (`cita_id`) en lugar de embeber toda la información del paciente.
- Desnormalización Obligatoria por Aislamiento de Motores: Se incluyó el campo `especialidad` (String) directamente en la raíz del documento de MongoDB, junto con `paciente_id` y `medico_id`. Aunque esto introduce un ligero *overhead* de almacenamiento frente a guardar solo el ID, es un requerimiento técnico estricto. Dado que el operador `$lookup` del *Aggregation Pipeline* de MongoDB no puede realizar uniones (*joins*) cruzadas hacia tablas alojadas en PostgreSQL, esta desnormalización es la única manera de permitir que MongoDB ejecute nativamente las consultas de agrupación por especialidad (RC-07 y RC-08) sin extraer y sobrecargar la memoria de la capa de aplicación en Node.js.
- Esquema Flexible (Embebido): Los datos médicos específicos se agruparon en arreglos y subdocumentos embebidos (`diagnosticos`, `medicamentos_recetados`, `datos_especificos_especialidad`). Este diseño de *One-to-Few* previene

operaciones \$lookup complejas, permitiendo recuperar el cuadro clínico completo de una consulta en una única operación de lectura.

4. Estrategia de Indexación

La creación de índices fue planificada para optimizar las operaciones críticas de lectura, midiendo el balance contra la penalización en las escrituras.

PostgreSQL:

1. Índice Único Parcial en citas (paciente_id, medico_id, fecha):
Se implementó un índice condicionado con la cláusula WHERE estado NOT IN ('cancelada', 'no_asistio'). Esto delega al motor la garantía de la RN-03 (un paciente no puede tener más de una cita activa con el mismo médico en el mismo día), pero permite correctamente que el paciente pueda volver a agendar en esa misma fecha si su cita original fue cancelada, asegurando la consistencia sin bloquear flujos de negocio válidos.
2. Índice B-Tree en facturas (estado): Acelera la generación de la vista de facturas pendientes (RC-02), la cual filtra constantemente los registros donde el estado difiera de 'pagada'.
3. Índice en pacientes (dpi): Mejora el rendimiento de las búsquedas transaccionales exactas al momento de atender a los pacientes en recepción.
4. Índice B-Tree compuesto en citas (medico_id, fecha): Acelera drásticamente la consulta de la agenda diaria (RC-01) y actúa como la base estructurada sobre la cual opera la validación de solapamiento de horarios (RN-01) dentro del *Stored Procedure* de creación/cancelación de citas (OC-02).

MongoDB:

1. Índice Único ({ cita_id: 1 }): Garantiza a nivel de motor de base de datos que no exista más de un historial clínico para la misma cita médica.
2. Índice Compuesto ({ especialidad: 1, "diagnosticos.codigo": 1 }): Diseñado específicamente para optimizar el *Pipeline* de Agregación que calcula el Top 5 de diagnósticos más frecuentes agrupados por especialidad.
3. Índice B-Tree en { paciente_id: 1 }: Permite una recuperación instantánea del historial clínico longitudinal completo de un individuo.

5. Estrategia de Vistas y Refresco de Datos (Refresh)

Para balancear la carga transaccional y analítica, se implementó una separación estricta mediante vistas en PostgreSQL:

- Vistas Normales (V1 y V2): Implementadas para la *Agenda Diaria* y las *Facturas Pendientes*. Se utilizan vistas normales ya que estos reportes operativos (RC-01 y RC-02) requieren acceso a información estrictamente en tiempo real para el personal de recepción.
- Vistas Materializadas (VM1 y VM2): Implementadas para los reportes gerenciales de *Ranking Trimestral de Médicos* y *Facturación Mensual*. La estrategia de *refresh* se programó en ventanas de mantenimiento de bajo impacto: VM1 se actualiza semanalmente (domingos a las 02:00 AM) y VM2 se actualiza el primer día de cada mes a las 03:00 AM.